EX NO : 1A **DATE :**	**Write a python program to compute the Central Tendency Measures : Mean, Median, Mode, Measures of Dispersion : Variance, Standard Deviation.**

AIM

To Write a python program to compute the Central Tendency Measures : Mean, Median, Mode, Measures of Dispersion : Variance, Standard Deviation.

ALGORITHM

Step 1 : Start the python program.

Step 2 : Import stat library.

Step 3 : Define the function for mean, median and mode.

Step 4 : Call these functions with the list of numbers.

Step 5 : The statistics function it calls the mean, median and mode and prints the result.

Step 6 : Stop the program.

PROGRAM

```
import statistics

def calculate_mean(data):
    return sum(data) / len(data)

def calculate_median(data):
    sorted_data = sorted(data)
    n = len(sorted_data)
    if n % 2 == 0:
        middle1 = sorted_data[n // 2 - 1]
        middle2 = sorted_data[n // 2]
        return (middle1 + middle2) / 2
    else:
        return sorted_data[n // 2]

def calculate_mode(data):
    return statistics.mode(data)

def calculate_variance(data):
    mean_value = calculate_mean(data)
    squared_diff_sum = sum((x - mean_value) ** 2 for x in data)
    return squared_diff_sum / (len(data) - 1)

def calculate_standard_deviation(data):
    variance_value = calculate_variance(data)
```

```
def calculate_mode(data):
    return statistics.mode(data)
def calculate_variance(data):
    mean_value = calculate_mean(data)
    squared_diff_sum = sum((x - mean_value) ** 2 for x in data)
    return squared_diff_sum / (len(data) - 1)
def calculate_standard_deviation(data):
    variance_value = calculate_variance(data)
    return variance_value ** 0.5
dataset = [10, 20, 30, 40, 50]
mean_value = calculate_mean(dataset)
median_value = calculate_median(dataset)
mode_value = calculate_mode(dataset)
variance_value = calculate_variance(dataset)
std_deviation_value = calculate_standard_deviation(dataset)
print(f"Dataset: {dataset}")
print(f"Mean: {mean_value:.2f}")
print(f"Median: {median_value:.2f}")
print(f"Mode: {mode_value}")
print(f"Variance: {variance_value:.2f}")
print(f"Standard Deviation: {std_deviation_value:.2f}")
```

OUTPUT

Dataset: [10, 20, 30, 40, 50]

Mean: 30.00

Median: 30.00

Mode: 10

Variance: 250.00

Standard Deviation: 15.81

EX NO : 1B DATE :	Sample Program

PROGRAM

```
import statistics

def compute_statistics(data):
    if not data:
        return "The data list is empty."
    try:
        mean=statistics.mean(data)
        median=statistics.median(data)
        mode=statistics.mode(data)
        variance=statistics.variance(data)
        std_dev=statistics.stdev(data)
        return{
            "Mean":mean,
            "Median":median,
            "Mode":mode,
            "Variance":variance,
            "Standard Deviation":std_dev
        }
```

```
    except statistics.StatisticsError as e:
        return f"An error occured:{e}"
data=[1,2,2,3,4,5,5,5,6,7,8]
statistics_result=compute_statistics(data)
for stat,value in statistics_result.items():
    print(f"{stat}:{value}")
```

OUTPUT

Mean:4.363636363636363

Median:5

Mode:5

Variance:4.8545454545454545

Standard Deviation:2.2033033051637387

RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 2A **DATE :**	**Implement a Linear Regression and Multiple Regression with Real Dataset.**

AIM

To implement a Linear Regression and Multiple Regression with Real Dataset.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Create a dataset by using MS Excel and save it as .csv.

Step 3 : Read the dataset into R.

Step 4 : Examine the structure and summary statistics of the dataset.

Step 5 : Plot the data to understand the relationship between variables.

Step 6 : Use the lm() for linear regression and mlm() for multiple regression to fit the model.

Step 7 : Obtain and interpret the summary of the fitted model.

Step 8 : Plot the regression along with the data.

Step 9 : Save the program by using .R extension and run the program.

DATASET

Lungcap	Age	Height	Smoke	Gender
40.24	9	64.5	No	Male
65.87	12	72.5	Yes	Female
86.65	14	83.7	No	Female
76.65	13	81.3	No	Male
75.7	13	12.97	No	Male
86.76	14	85.75	No	Male
58.78	10	64.76	No	Male
66.76	10	63.9	No	Female
86.87	13	60.8	No	Male
77.75	11	58.8	No	Male

PROGRAM

LINEAR REGRESSION

```
lung<-read.csv("C:/Users/ELCOT/Documents/Rstudio/lungcapdata.csv")

names(lungcapdata)

class('Age')

head(lungcapdata)

lungcapdata[1:3]

dim(lungcapdata)

summary(lungcapdata,maxsum=5)

plot(x=lungcapdata $Lungcap,

    y=lungcapdata $Height,

    xlab="Lungcap",
```

```
    y="Height",
    col="red",
    main="Lungcap Vs Height")
mod<-lm(Lungcap~Height,data=lungcapdata)
summary(mod)
plot(lungcapdata $Lungcap,lungcapdata $Height)
abline(mod)
```

OUTPUT

```
> lung<-read.csv("C:/Users/ELCOT/Documents/Rstudio/lungcapdata.csv")

> names(lungcapdata)
[1] "Lungcap" "Age"     "Height"  "Smoke"   "Gender"

> class('Age')
[1] "character"

> head(lungcapdata)
  Lungcap Age Height Smoke Gender
1   40.24   9  64.50    No   Male
2   65.87  12  72.50   Yes Female
3   86.65  14  83.70    No Female
4   76.65  13  81.30    No   Male
5   75.70  13  12.97    No   Male
6   86.76  14  85.75    No   Male

> lungcapdata[1:3]
  Lungcap Age Height
1   40.24   9  64.50
2   65.87  12  72.50
3   86.65  14  83.70
4   76.65  13  81.30
5   75.70  13  12.97
6   86.76  14  85.75
7   58.78  10  64.76
8   66.76  10  63.90
9   86.87  13  60.80
10  77.75  11  58.80
```

```
> dim(lungcapdata)
[1] 10  5

> summary(lungcapdata,maxsum=5)
    Lungcap          Age           Height         Smoke              Gender
 Min.   :40.24   Min.   : 9.00   Min.   :12.97   Length:10          Length:10
 1st Qu.:66.09   1st Qu.:10.25   1st Qu.:61.58   Class :character   Class :character
 Median :76.17   Median :12.50   Median :64.63   Mode  :character   Mode  :character
 Mean   :72.20   Mean   :11.90   Mean   :64.90
 3rd Qu.:84.42   3rd Qu.:13.00   3rd Qu.:79.10
 Max.   :86.87   Max.   :14.00   Max.   :85.75
> plot(x=lungcapdata $Lungcap, y=lungcapdata $Height, xlab="Lungcap", y="Height",
col="red", main="Lungcap Vs Height")

> mod<-lm(Lungcap~Height,data=lungcapdata)

> summary(mod)

Call:
lm(formula = Lungcap ~ Height, data = lungcapdata)

Residuals:
    Min      1Q  Median      3Q     Max
-31.928  -6.596   4.538  11.555  15.032

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 66.42191   16.97910   3.912  0.00447 **
Height       0.08908    0.25039   0.356  0.73121
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

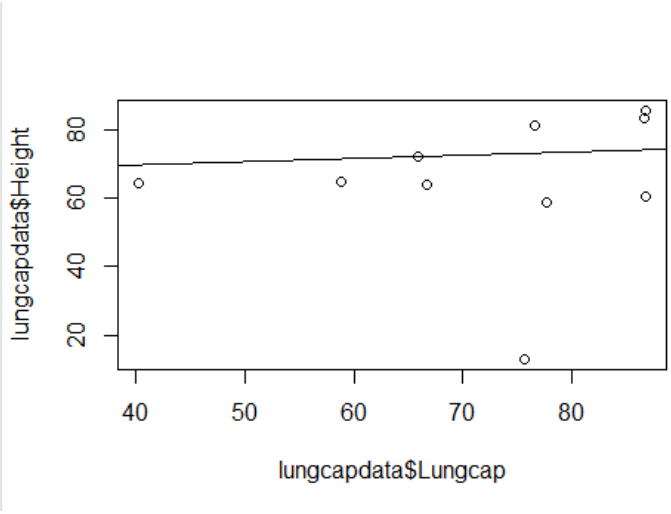
Residual standard error: 15.57 on 8 degrees of freedom

Multiple R-squared: 0.01558, Adjusted R-squared: -0.1075

F-statistic: 0.1266 on 1 and 8 DF, p-value: 0.7312

> plot(lungcapdata $Lungcap,lungcapdata $Height)

> abline(mod)



MULTIPLE REGRESSION

```
lung<-read.csv("C:/Users/ELCOT/Documents/Rstudio/lungcapdata.csv")

names(lungcapdata)

class('Age')

head(lungcapdata)

lungcapdata[1:3]

dim(lungcapdata)

summary(lungcapdata,maxsum=5)

plot(x=lungcapdata $Lungcap,

    y=lungcapdata $Height,

    xlab="Lungcap", y="Height", col="red", main="Lungcap Vs Height")

mod<-lm(Lungcap~Height,data=lungcapdata)

summary(mod)

cor(Age,Height,method="pearson")

confint(mod,conf.level=0.95)

mod1<-lm(Lungcap~Age+Smoke,data=lungcapdata)

summary(mod1)

plot(lungcapdata $Lungcap,lungcapdata $Height)

abline(mod)

plot(mod1)
```

OUTPUT

```
> lung<-read.csv("C:/Users/ELCOT/Documents/Rstudio/lungcapdata.csv")

> names(lungcapdata)
[1] "Lungcap" "Age"     "Height"  "Smoke"   "Gender"

> class('Age')
[1] "character"

> head(lungcapdata)
  Lungcap Age Height Smoke Gender
1   40.24   9  64.50    No   Male
2   65.87  12  72.50   Yes Female
3   86.65  14  83.70    No Female
4   76.65  13  81.30    No   Male
5   75.70  13  12.97    No   Male
6   86.76  14  85.75    No   Male

> lungcapdata[1:3]
  Lungcap Age Height
1   40.24   9  64.50
2   65.87  12  72.50
3   86.65  14  83.70
4   76.65  13  81.30
5   75.70  13  12.97
6   86.76  14  85.75
7   58.78  10  64.76
8   66.76  10  63.90
9   86.87  13  60.80
10  77.75  11  58.80
```

```
> dim(lungcapdata)
[1] 10  5

> summary(lungcapdata,maxsum=5)
    Lungcap          Age           Height         Smoke              Gender
 Min.   :40.24   Min.   : 9.00   Min.   :12.97   Length:10          Length:10
 1st Qu.:66.09   1st Qu.:10.25   1st Qu.:61.58   Class :character   Class :character
 Median :76.17   Median :12.50   Median :64.63   Mode  :character   Mode  :character
 Mean   :72.20   Mean   :11.90   Mean   :64.90
 3rd Qu.:84.42   3rd Qu.:13.00   3rd Qu.:79.10
 Max.   :86.87   Max.   :14.00   Max.   :85.75
> plot(x=lungcapdata $Lungcap, y=lungcapdata $Height, xlab="Lungcap", y="Height",
col="red", main="Lungcap Vs Height")

> mod<-lm(Lungcap~Height,data=lungcapdata)

> summary(mod)

Call:
lm(formula = Lungcap ~ Height, data = lungcapdata)

Residuals:
    Min      1Q  Median      3Q     Max
-31.928  -6.596   4.538  11.555  15.032

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 66.42191   16.97910   3.912  0.00447 **
Height       0.08908    0.25039   0.356  0.73121
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

```
Residual standard error: 15.57 on 8 degrees of freedom
Multiple R-squared:  0.01558,           Adjusted R-squared:  -0.1075
F-statistic: 0.1266 on 1 and 8 DF,  p-value: 0.7312

> cor(Age,Height,method="pearson")

> confint(mod,conf.level=0.95)
                2.5 %      97.5 %
(Intercept) 27.2680324 105.5757837
Height      -0.4883112   0.6664705

> mod1<-lm(Lungcap~Age+Smoke,data=lungcapdata)

> summary(mod1)

Call:
lm(formula = Lungcap ~ Age + Smoke, data = lungcapdata)

Residuals:
    Min      1Q  Median      3Q     Max
-11.727  -3.623  -0.942   4.432  11.286

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -13.268     17.072  -0.777  0.46250
Age            7.248      1.420   5.105  0.00139 **
SmokeYes      -7.842      8.046  -0.975  0.36222
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

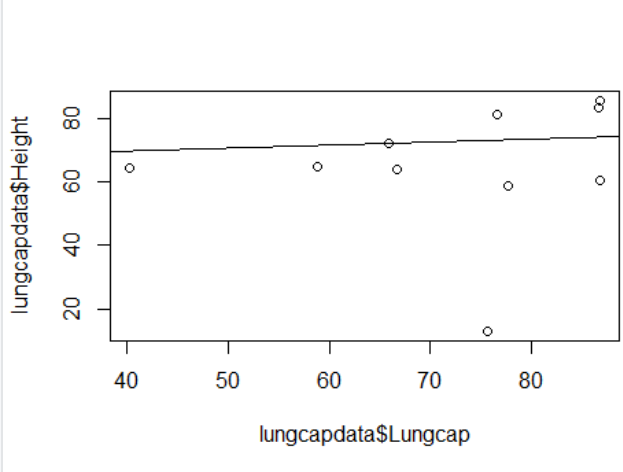
Residual standard error: 7.632 on 7 degrees of freedom

Multiple R-squared: 0.793, Adjusted R-squared: 0.7339

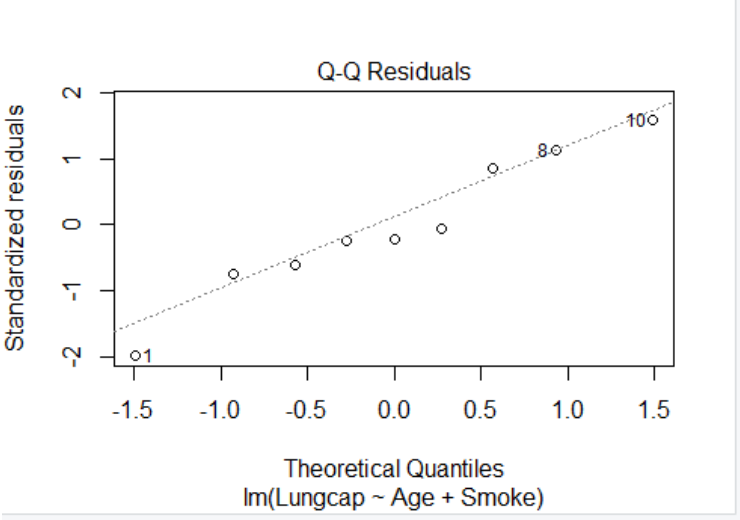
F-statistic: 13.41 on 2 and 7 DF, p-value: 0.004033

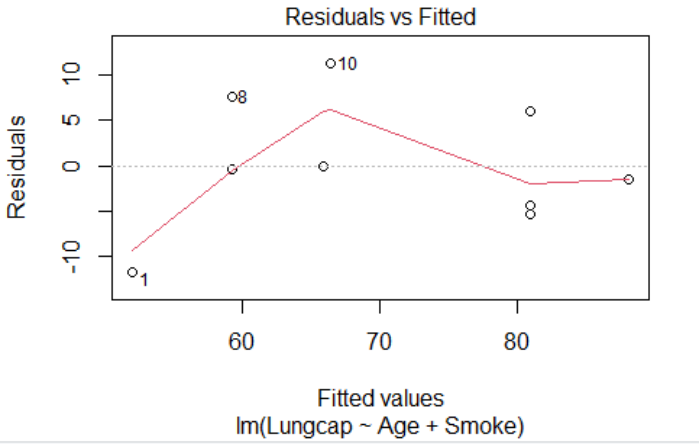
```
> plot(lungcapdata $Lungcap,lungcapdata $Height)
```

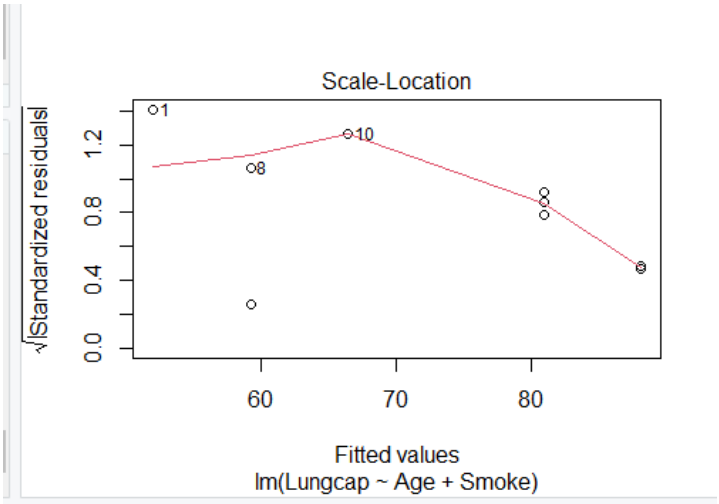
```
> abline(mod)
```

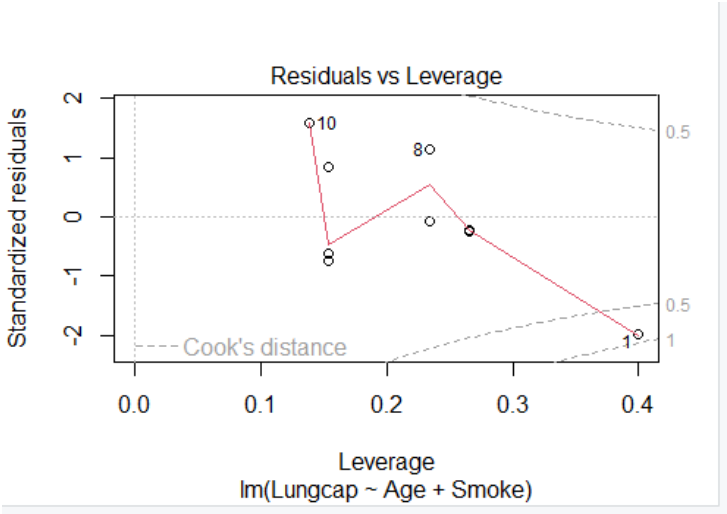



```
plot(mod1)
```




Residuals vs Fitted
Residuals
10
5
0
-10
10
8
1
60
70
80
Fitted values
lm(Lungcap ~ Age + Smoke)


Scale-Location
√|Standardized residuals|
0.0
0.4
0.8
1.2
1
10
8
60
70
80
Fitted values
lm(Lungcap ~ Age + Smoke)


Residuals vs Leverage
Standardized residuals
2
1
0
-1
-2
10
8
1
0.5
0.5
1
Cook's distance
0.0
0.1
0.2
0.3
0.4
Leverage
lm(Lungcap ~ Age + Smoke)

EX NO : 2B **DATE :**	**Sample Program**

DATASET

Medv	Rm	age	dis
24	6.32	65.2	4.09
21.6	7.07	78.9	4.967
34.7	7.07	61.1	4.967
33.4	7.07	45.8	4.967
36.2	7.07	54.2	4.967
28.7	6.42	58.7	5.188
22.9	66.67	66.6	5.522
27.1	6.17	47.8	5.091
16.5	6.33	77.8	4.967
18.9	6.01	61.9	5.188
15.3	5.92	45.9	6.062
14.4	6.08	54.4	5.69
24.5	6.38	58.3	4.967

PROGRAM

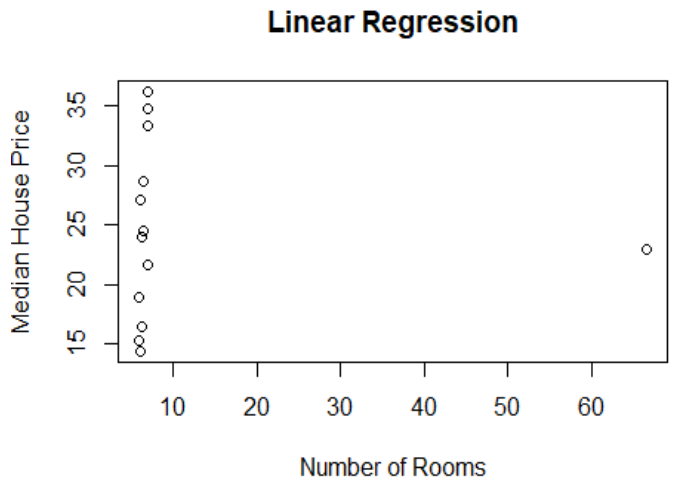
LINEAR REGRESSION

```
medv<-read.csv("C:/Users/ELCOT/Desktop/Boston.csv")

names(Boston)

class('rm')

head(Boston)

Boston[1:3]

summary(Boston,maxsum=5)

plot(Boston$rm, Boston$medv, main = "Linear Regression", xlab = "Number of Rooms", ylab = "Median House Price")

lm_model <- lm(medv ~ rm, data = Boston)

summary(lm_model)

abline(lm_model, col = "red")
```

OUTPUT

```
> medv<-read.csv("C:/Users/ELCOT/Desktop/Boston.csv")
> names(Boston)
[1] "medv" "rm"   "age"  "dis"
> class('rm')
[1] "character"
> head(Boston)
  medv   rm  age   dis
1   NA   NA   NA    NA
2 24.0 6.32 65.2 4.090
3 21.6 7.07 78.9 4.967
4 34.7 7.07 61.1 4.967
5 33.4 7.07 45.8 4.967
6 36.2 7.07 54.2 4.967
> Boston[1:3]
   medv    rm  age
1    NA    NA   NA
2  24.0  6.32 65.2
3  21.6  7.07 78.9
4  34.7  7.07 61.1
5  33.4  7.07 45.8
6  36.2  7.07 54.2
7  28.7  6.42 58.7
8  22.9 66.67 66.6
9  27.1  6.17 47.8
10 16.5  6.33 77.8
11 18.9  6.01 61.9
12 15.3  5.92 45.9
13 14.4  6.08 54.4
14 24.5  6.38 58.3
```

```
> summary(Boston,maxsum=5)
      medv            rm            age             dis
 Min.   :14.40   Min.   : 5.92   Min.   :45.80   Min.   :4.090
 1st Qu.:18.90   1st Qu.: 6.17   1st Qu.:54.20   1st Qu.:4.967
 Median :24.00   Median : 6.38   Median :58.70   Median :4.967
 Mean   :24.48   Mean   :11.12   Mean   :59.74   Mean   :5.126
 3rd Qu.:28.70   3rd Qu.: 7.07   3rd Qu.:65.20   3rd Qu.:5.188
 Max.   :36.20   Max.   :66.67   Max.   :78.90   Max.   :6.062
 NA's   :1       NA's   :1       NA's   :1       NA's   :1
> plot(Boston$rm, Boston$medv, main = "Linear Regression", xlab = "Number of Rooms", ylab =
"Median House Price")
```




```
> lm_model <- lm(medv ~ rm, data = Boston)

> summary(lm_model)

Call:
lm(formula = medv ~ rm, data = Boston)

Residuals:
     Min       1Q   Median       3Q      Max
```

-10.1770 -5.6784 -0.4742   4.1297 11.6426

Coefficients:

	Estimate	Std. Error	t value	Pr(>|t|)	
(Intercept)	24.69771	2.56712	9.621	1.09e-06	***
rm	-0.01985	0.13152	-0.151	0.883	

Signif. codes: 0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

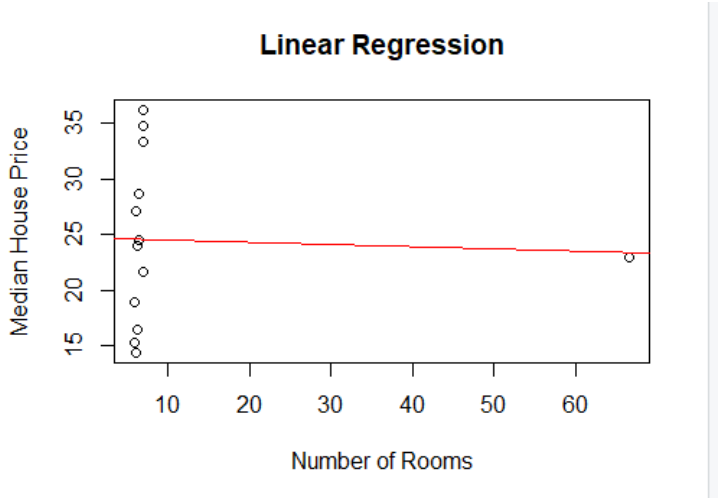
Residual standard error: 7.606 on 11 degrees of freedom

(1 observation deleted due to missingness)

Multiple R-squared: 0.002067, Adjusted R-squared: -0.08865

F-statistic: 0.02278 on 1 and 11 DF, p-value: 0.8828

```
> abline(lm_model, col = "red")
```



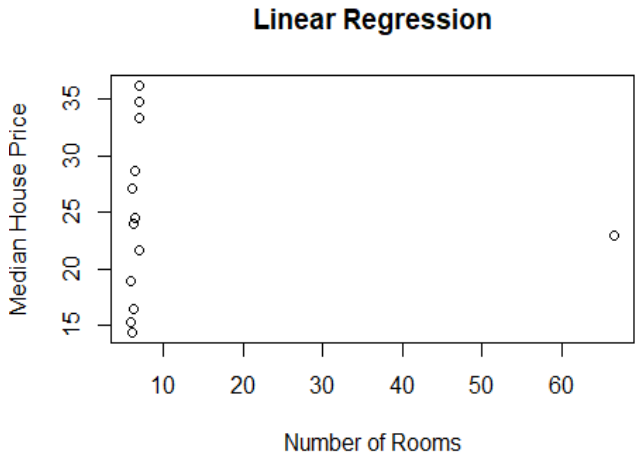
MULTIPLE REGRESSION

```
medv<-read.csv("C:/Users/ELCOT/Desktop/Boston.csv")

names(Boston)

class('rm')

head(Boston)

Boston[1:3]

summary(Boston,maxsum=5)

plot(Boston$rm, Boston$medv, main = "Linear Regression", xlab = "Number of Rooms", ylab = "Median House Price")

lm_model <- lm(medv ~ rm, data = Boston)

summary(lm_model)

mlm_model <- lm(medv ~ rm + age + dis, data = Boston)

summary(mlm_model)

plot(Boston$rm, Boston$medv, main = "Multiple Regression - Rooms", xlab = "Number of Rooms", ylab = "Median House Price")

abline(lm_model, col = "red")

plot(mlm_model)
```

OUTPUT

```
> medv<-read.csv("C:/Users/ELCOT/Desktop/Boston.csv")
> names(Boston)
[1] "medv" "rm"   "age"  "dis"
> class('rm')
[1] "character"
> head(Boston)
  medv   rm  age   dis
1   NA   NA   NA    NA
2 24.0 6.32 65.2 4.090
3 21.6 7.07 78.9 4.967
4 34.7 7.07 61.1 4.967
5 33.4 7.07 45.8 4.967
6 36.2 7.07 54.2 4.967
> Boston[1:3]
   medv    rm  age
1    NA    NA   NA
2  24.0  6.32 65.2
3  21.6  7.07 78.9
4  34.7  7.07 61.1
5  33.4  7.07 45.8
6  36.2  7.07 54.2
7  28.7  6.42 58.7
8  22.9 66.67 66.6
9  27.1  6.17 47.8
10 16.5  6.33 77.8
11 18.9  6.01 61.9
12 15.3  5.92 45.9
13 14.4  6.08 54.4
14 24.5  6.38 58.3
```

```
> summary(Boston,maxsum=5)
      medv            rm            age             dis
 Min.   :14.40   Min.   : 5.92   Min.   :45.80   Min.   :4.090
 1st Qu.:18.90   1st Qu.: 6.17   1st Qu.:54.20   1st Qu.:4.967
 Median :24.00   Median : 6.38   Median :58.70   Median :4.967
 Mean   :24.48   Mean   :11.12   Mean   :59.74   Mean   :5.126
 3rd Qu.:28.70   3rd Qu.: 7.07   3rd Qu.:65.20   3rd Qu.:5.188
 Max.   :36.20   Max.   :66.67   Max.   :78.90   Max.   :6.062
 NA's   :1       NA's   :1       NA's   :1       NA's   :1
> plot(Boston$rm, Boston$medv, main = "Linear Regression", xlab = "Number of Rooms", ylab =
"Median House Price")
```




```
> lm_model <- lm(medv ~ rm, data = Boston)

> summary(lm_model)
Call:
lm(formula = medv ~ rm, data = Boston)
Residuals:
    Min      1Q  Median      3Q     Max
-10.1770  -5.6784  -0.4742   4.1297  11.6426
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 24.69771    2.56712   9.621 1.09e-06 ***
```

rm -0.01985 0.13152 -0.151 0.883

Signif. codes: 0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

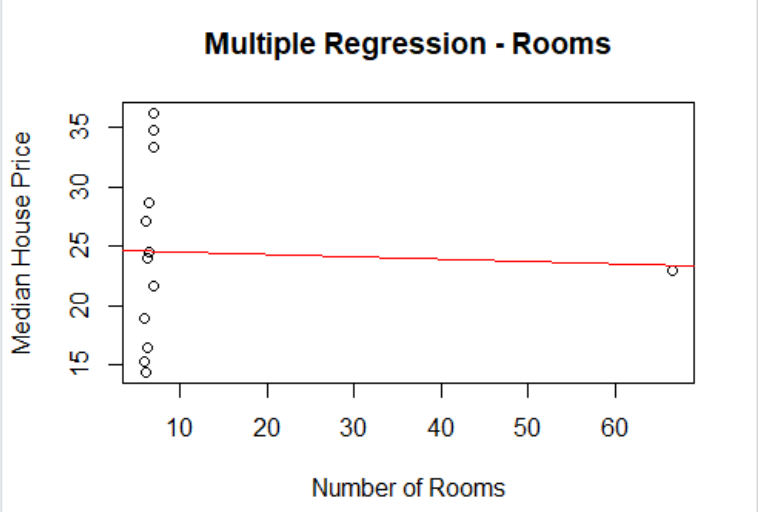
Residual standard error: 7.606 on 11 degrees of freedom

(1 observation deleted due to missingness)

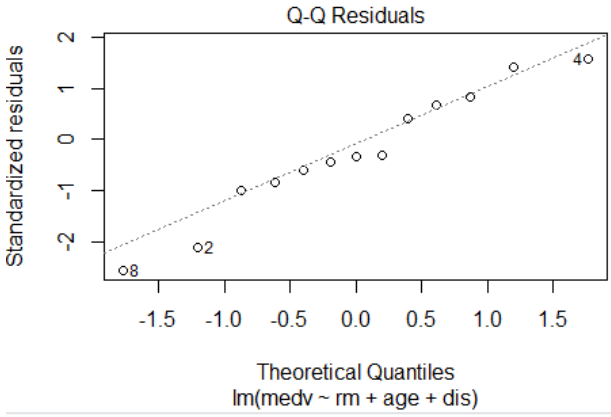
Multiple R-squared: 0.002067, Adjusted R-squared: -0.08865

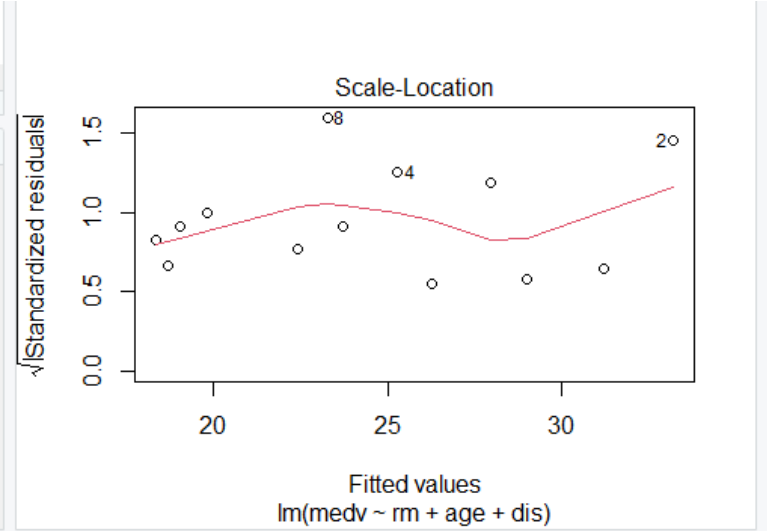
F-statistic: 0.02278 on 1 and 11 DF, p-value: 0.8828

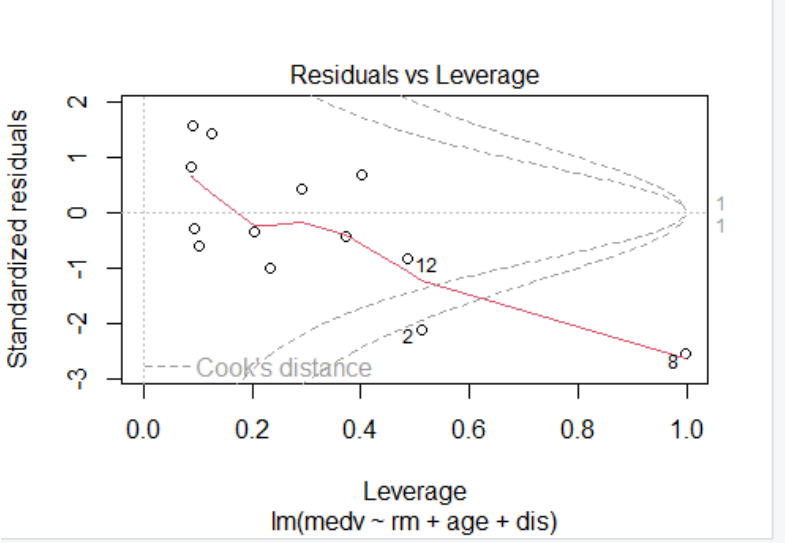
```
> plot(Boston$rm, Boston$medv, main = "Multiple Regression - Rooms", xlab = "Number of Rooms",
ylab = "Median House Price")
> abline(lm_model, col = "red")
```




```
> plot(mlm_model)
```







RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 3A **DATE :**	**Implementation of Logistic Regression using Sklearn.**

AIM

To create a python program for Implementing the Logistic Regression using Sklearn.

ALGORITHM

Step 1 : Open the python platform.

Step 2 : Import Necessary library.

Step 3 : Load your dataset and selecting feature.

Step 4 : Splitting data into feature(X) and target(Y).

Step 5 : Train the model development and prediction.

Step 6 : evaluate the model using Confusion Matrix.

Step 7 : Visualizing confusion matrix using Heatmap.

Step 8 : Confusion Matrix evaluation metrics.

PROGRAM

```
from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn import metrics

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

diab_df = pd.read_csv("diabetes.csv")

diab_df.head()

diab_cols = ['Pregnancies', 'Insulin', 'BMI', 'Age','Glucose','BloodPressure','DiabetesPedigreeFunction']

X = diab_df[diab_cols]# Features

y = diab_df.Outcome # Target variable

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_state=0)

logreg =  LogisticRegression(solver='liblinear')

logreg.fit(X_train,y_train)

y_pred=logreg.predict(X_test)

y_pred

cnf_matrix = metrics.confusion_matrix(y_test, y_pred)

cnf_matrix

class_names=[0,1]
```

```
fig, ax = plt.subplots()

tick_marks = np.arange(len(class_names))

plt.xticks(tick_marks, class_names)

plt.yticks(tick_marks, class_names)

sns.heatmap(pd.DataFrame(cnf_matrix), annot=True, cmap="YlGnBu" ,fmt='g')

ax.xaxis.set_label_position("top")

plt.tight_layout()

plt.title('Confusion matrix', y=1.1)

plt.ylabel('Actual label')

plt.xlabel('Predicted label')

print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

print("Precision:",metrics.precision_score(y_test, y_pred))

print("Recall:",metrics.recall_score(y_test, y_pred))
```

OUTPUT

Accuracy: 0.8072916666666666

Precision: 0.7659574468085106

Recall: 0.5806451612903226

EX NO : 3B DATE :	Sample Program

PROGRAM

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix
data={'sepal length':[5.1,4.9,4.7,4.6,5],
    'sepal width':[3.5,3.0,3.2,3.1,3.6],
    'petal length':[1.4,1.4,1.3,1.5,1.4],
    'petal width':[0.2,0.2,0.2,0.2,0.2],
    'target':[0,0,0,0,0]
    }
df=pd.DataFrame(data)
x=df[['sepal length','sepal width','petal length','petal width']]
y=df['target']
iris=load_iris()
x=iris.data
y=iris.target
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

logreg=LogisticRegression(max_iter=10000)

logreg.fit(x_train,y_train)

y_pred=logreg.predict(x_test)

accuracy=accuracy_score(y_test,y_pred)

print("Accuracy:",accuracy)

print("Classification Report:")

print(classification_report(y_test,y_pred))

print("Confusion Matrix:")

print(confusion_matrix(y_test,y_pred))
```

OUTPUT

Accuracy : 1.0

Classification Report:

	precision	recall	f-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
Accuracy				
Macro avg	1.00	1.00	1.00	30
Weighted avg	1.00	1.00	1.00	30

Confusion Matrix:

[[10,0,0]

[0,9,0]

[0,0,11]]

RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 4A **DATE :**	**Implement a Binary Classification model.**

AIM

To implement a binary classification model by using dataset.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Create a dataset by using MS Excel and save it as .csv.

Step 3 : Read the default dataset iris into R.

Step 4 : Examine the structure and summary statistics of the dataset.

Step 5 : Install the libraries caret, dplyr, ggplot2.

Step 6 : Use the glm() to fit the model.

Step 7 : Obtain and interpret the summary of the fitted model.

Step 8 : Plot the binary classification model along with the data.

Step 9 : Save the program by using .R extension and run the program.

PROGRAM

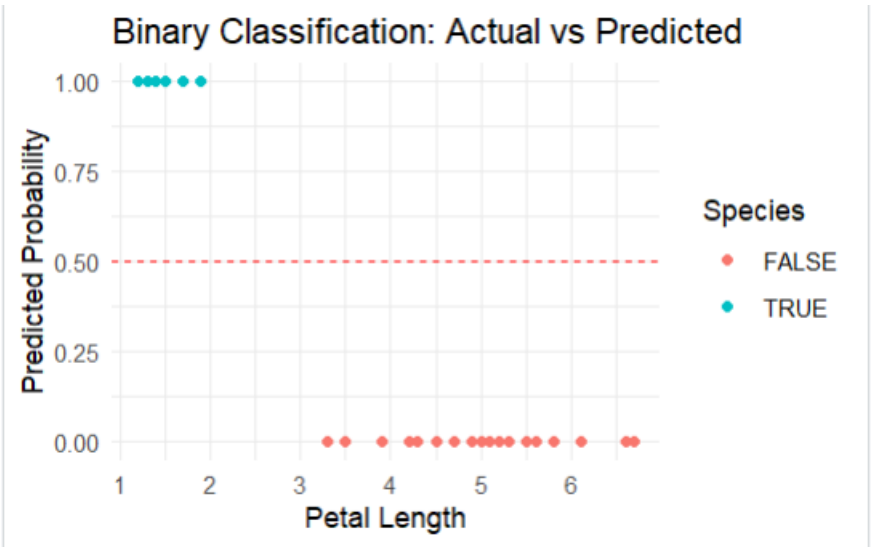
```
library(caret)

library(dplyr)

library(ggplot2)

data(iris)

iris$Species<- factor(iris$Species == "setosa")

summary(iris)

head(iris)

set.seed(123)

index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)

train_data<- iris[index, ]

test_data<- iris[-index, ]

model <- glm(Species ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,

        data = train_data, family = binomial)

predictions <- predict(model, newdata = test_data, type = "response")

predictions_binary<- ifelse(predictions > 0.5, 1, 0)

confusion_matrix<- table(Actual = test_data$Species, Predicted = predictions_binary)

print(confusion_matrix)

accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)

print(accuracy)

ggplot(test_data, aes(x = Petal.Length, y = predictions, color = Species)) +  geom_point() +
```

```
labs(title = "Binary Classification: Actual vs Predicted",
     x = "Petal Length",
     y = "Predicted Probability") +
geom_hline(yintercept = 0.5, linetype = "dashed", color = "red") +
theme_minimal()
```

OUTPUT

```
> library(caret)
> library(dplyr)
> library(ggplot2)
> data(iris)
> iris$Species<- factor(iris$Species == "setosa")
> summary(iris)
 Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
Median :5.800   Median :3.000   Median :4.350   Median :1.300
Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
 Species
FALSE:100
TRUE : 50
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2    TRUE
2          4.9         3.0          1.4         0.2    TRUE
3          4.7         3.2          1.3         0.2    TRUE
4          4.6         3.1          1.5         0.2    TRUE
5          5.0         3.6          1.4         0.2    TRUE
6          5.4         3.9          1.7         0.4    TRUE
> set.seed(123)
> index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
> train_data<- iris[index, ]
> test_data<- iris[-index, ]
> model <- glm(Species ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,
+              data = train_data, family = binomial)
```

```
> predictions <- predict(model, newdata = test_data, type = "response")
> predictions_binary<- ifelse(predictions > 0.5, 1, 0)
> confusion_matrix<- table(Actual = test_data$Species, Predicted = predictions_binary)
> print(confusion_matrix)
       Predicted
Actual   0  1
  FALSE 30  0
  TRUE   0 15
> accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
> print(accuracy)
[1] 1
> ggplot(test_data, aes(x = Petal.Length, y = predictions, color = Species)) +
+   geom_point() +
+   labs(title = "Binary Classification: Actual vs Predicted",
+        x = "Petal Length",
+        y = "Predicted Probability") +
+   geom_hline(yintercept = 0.5, linetype = "dashed", color = "red") +
+   theme_minimal()
```



EX NO : 4B DATE :	Sample Program

PROGRAM

```
library(caret)

library(rpart)

library(ggplot2)

data(iris)

iris$Species <- factor(iris$Species == "setosa")

set.seed(123)

index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)

train_data <- iris[index, ]

test_data <- iris[-index, ]

model <- rpart(Species ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,

        data = train_data,

        method = "class")


predictions <- predict(model, newdata = test_data, type = "class")

conf_matrix <- confusionMatrix(predictions, test_data$Species)

print(conf_matrix)

ggplot(test_data, aes(x = Petal.Length, y = Petal.Width, color = predictions)) +
```

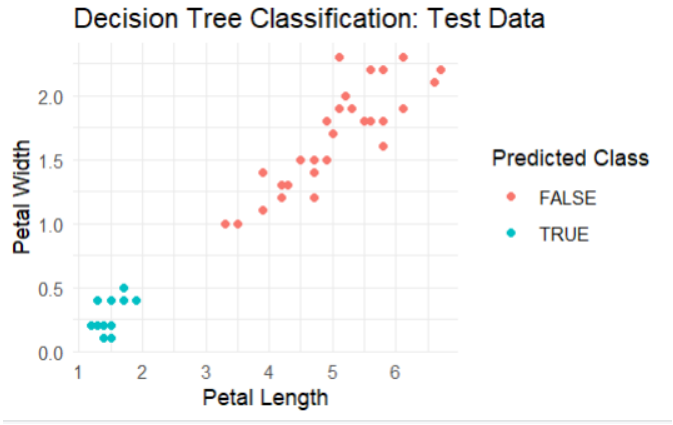
```
geom_point() +
labs(title = "Decision Tree Classification: Test Data",
     x = "Petal Length",
     y = "Petal Width",
     color = "Predicted Class") +
theme_minimal()
```

OUTPUT

```
> library(caret)
> library(rpart)
> library(ggplot2)
> data(iris)
> iris$Species <- factor(iris$Species == "setosa")
> set.seed(123)
> index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
> train_data <- iris[index, ]
> test_data <- iris[-index, ]
> model <- rpart(Species ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,
+                data = train_data,
+                method = "class")
> predictions <- predict(model, newdata = test_data, type = "class")
> conf_matrix <- confusionMatrix(predictions, test_data$Species)
> print(conf_matrix)
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE    30    0
     TRUE      0   15

               Accuracy : 1
                 95% CI : (0.9213, 1)
    No Information Rate : 0.6667
    P-Value [Acc > NIR] : 1.191e-08

                  Kappa : 1

 Mcnemar's Test P-Value : NA
```

```
            Sensitivity : 1.0000
            Specificity : 1.0000
         Pos Pred Value : 1.0000
         Neg Pred Value : 1.0000
             Prevalence : 0.6667
         Detection Rate : 0.6667
   Detection Prevalence : 0.6667
      Balanced Accuracy : 1.0000
       'Positive' Class : FALSE
```

```
> ggplot(test_data, aes(x = Petal.Length, y = Petal.Width, color = predictions)) +
+   geom_point() +
+   labs(title = "Decision Tree Classification: Test Data",
+        x = "Petal Length",
+        y = "Petal Width",
+        color = "Predicted Class") +
+   theme_minimal()
>
```



RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 5A **DATE :**	**Classification with Nearest Neighbours and Naïve Baye Algorithm.**

AIM

To implement a classification with with Nearest Neighbours and Naïve Baye Algorithm.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Create a dataset by using MS Excel and save it as .csv.

Step 3 : Read the default dataset iris into R.

Step 4 : Examine the structure and summary statistics of the dataset.

Step 5 : Install the libraries class, e1071, caret, ggplot2, girdExtra.

Step 6 : Use the knn() to fit the model.

Step 7 : Obtain and interpret the summary of the fitted model.

Step 8 : Plot the classification with nearest neighbours and naïve baye algorithm model along with the data.

Step 9 : Save the program by using .R extension and run the program.

PROGRAM

```
library(class)

library(e1071)

library(caret)

library(ggplot2)

library(gridExtra)

data("iris")

head(iris)

iris$Species<- factor(iris$Species)

set.seed(123)

index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)

train_data<- iris[index, ]

test_data<- iris[-index, ]

train_features<- train_data[, -5]

train_labels<- train_data$Species

test_features<- test_data[, -5]

test_labels<- test_data$Species

knn_predictions<- knn(train = train_features, test = test_features, cl = train_labels, k = k)

confusion_matrix_knn<- table(Predicted = knn_predictions, Actual = test_labels)

print("k-NN Confusion Matrix:")

print(confusion_matrix_knn)
```

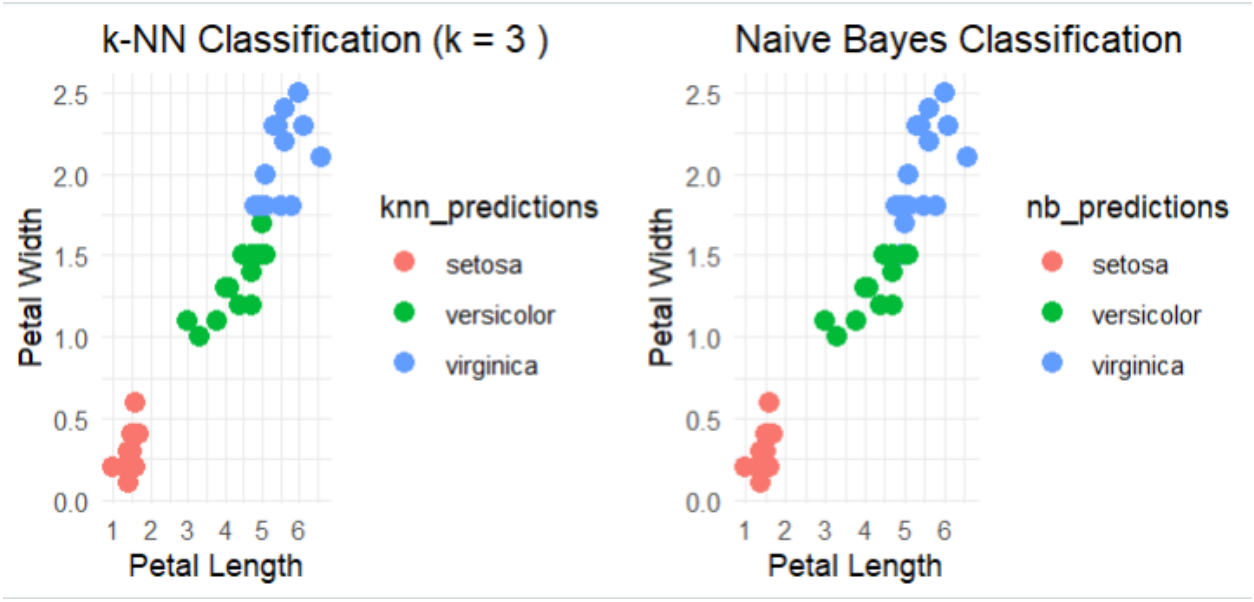
```
accuracy_knn<- sum(diag(confusion_matrix_knn)) / sum(confusion_matrix_knn)
print(paste("k-NN Accuracy:", accuracy_knn))
plot_knn<- ggplot(test_data, aes(Petal.Length, Petal.Width, color = knn_predictions)) +
  geom_point(size = 3) +
  labs(title = paste("k-NN Classification (k =", k, ")"),
       x = "Petal Length",
       y = "Petal Width") +
  theme_minimal()
nb_model<- naiveBayes(Species ~ ., data = train_data)
nb_predictions<- predict(nb_model, newdata = test_data)
confusion_matrix_nb<- table(Predicted = nb_predictions, Actual = test_labels)
print("Naive Bayes Confusion Matrix:")
print(confusion_matrix_nb)
accuracy_nb<- sum(diag(confusion_matrix_nb)) / sum(confusion_matrix_nb)
print(paste("Naive Bayes Accuracy:", accuracy_nb))
plot_nb<- ggplot(test_data, aes(Petal.Length, Petal.Width, color = nb_predictions)) +
  geom_point(size = 3) +
  labs(title = "Naive Bayes Classification",
       x = "Petal Length",
       y = "Petal Width") +  theme_minimal()
grid.arrange(plot_knn, plot_nb, ncol = 2)
```

OUTPUT

```
> library(class)
> library(e1071)
> library(caret)
> library(ggplot2)
> library(gridExtra)
> data("iris")
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
> iris$Species<- factor(iris$Species)
> set.seed(123)
> index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
> train_data<- iris[index, ]
> test_data<- iris[-index, ]
> train_features<- train_data[, -5]
> train_labels<- train_data$Species
> test_features<- test_data[, -5]
> test_labels<- test_data$Species
> knn_predictions<- knn(train = train_features, test = test_features, cl = train_labels, k = k)
> confusion_matrix_knn<- table(Predicted = knn_predictions, Actual = test_labels)
> print("k-NN Confusion Matrix:")
[1] "k-NN Confusion Matrix:"
> print(confusion_matrix_knn)
            Actual
Predicted    setosa versicolor virginica
```

```
  setosa         15         0         0
  versicolor      0         15        2
  virginica       0         0        13
> accuracy_knn<- sum(diag(confusion_matrix_knn)) / sum(confusion_matrix_knn)
> print(paste("k-NN Accuracy:", accuracy_knn))
[1] "k-NN Accuracy: 0.955555555555556"
> plot_knn<- ggplot(test_data, aes(Petal.Length, Petal.Width, color = knn_predictions)) +
+   geom_point(size = 3) +
+   labs(title = paste("k-NN Classification (k =", k, ")"),
+        x = "Petal Length",
+        y = "Petal Width") +
+   theme_minimal()
> nb_model<- naiveBayes(Species ~ ., data = train_data)
> nb_predictions<- predict(nb_model, newdata = test_data)
> confusion_matrix_nb<- table(Predicted = nb_predictions, Actual = test_labels)
> print("Naive Bayes Confusion Matrix:")
[1] "Naive Bayes Confusion Matrix:"
> print(confusion_matrix_nb)
            Actual
Predicted    setosa versicolor virginica
  setosa         15         0         0
  versicolor      0         13        2
  virginica       0         2        13
> accuracy_nb<- sum(diag(confusion_matrix_nb)) / sum(confusion_matrix_nb)
> print(paste("Naive Bayes Accuracy:", accuracy_nb))
[1] "Naive Bayes Accuracy: 0.911111111111111"
> plot_nb<- ggplot(test_data, aes(Petal.Length, Petal.Width, color = nb_predictions)) +
+   geom_point(size = 3) +
+   labs(title = "Naive Bayes Classification",
+        x = "Petal Length",
+        y = "Petal Width") +
```

```
+   theme_minimal()
> grid.arrange(plot_knn, plot_nb, ncol = 2)
```



EX NO : 5B DATE :	Sample Program

PROGRAM

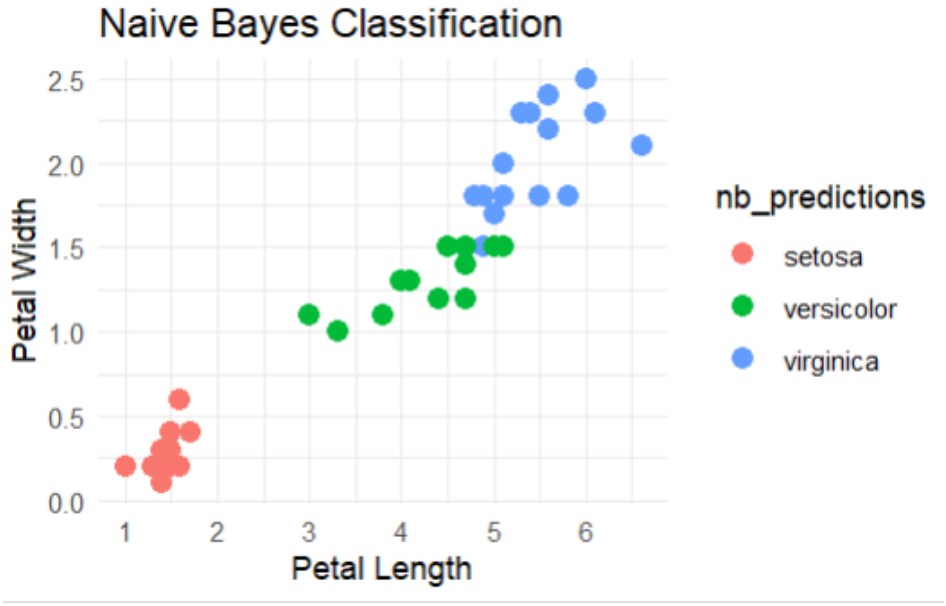
```
library(e1071)

library(caret)

library(ggplot2)

library(gridExtra)

data("iris")

head(iris)

iris$Species <- factor(iris$Species)

set.seed(123)

index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)

train_data <- iris[index, ]

test_data <- iris[-index, ]

nb_model <- naiveBayes(Species ~ ., data = train_data)

nb_predictions <- predict(nb_model, newdata = test_data)

confusion_matrix_nb <- table(Predicted = nb_predictions, Actual = test_data$Species)

print("Naive Bayes Confusion Matrix:")

print(confusion_matrix_nb)

accuracy_nb <- sum(diag(confusion_matrix_nb)) / sum(confusion_matrix_nb)
```

```
print(paste("Naive Bayes Accuracy:", accuracy_nb))

plot_nb <- ggplot(test_data, aes(Petal.Length, Petal.Width, color = nb_predictions)) +

  geom_point(size = 3) +

  labs(title = "Naive Bayes Classification",

       x = "Petal Length",

       y = "Petal Width") +

  theme_minimal()

print(plot_nb)
```

OUTPUT

```
> library(e1071)
> library(caret)
> library(ggplot2)
> library(gridExtra)
> data("iris")
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
> iris$Species <- factor(iris$Species)
> set.seed(123)
> index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
> train_data <- iris[index, ]
> test_data <- iris[-index, ]
> nb_model <- naiveBayes(Species ~ ., data = train_data)
> nb_predictions <- predict(nb_model, newdata = test_data)
> confusion_matrix_nb <- table(Predicted = nb_predictions, Actual = test_data$Species)
> print("Naive Bayes Confusion Matrix:")
[1] "Naive Bayes Confusion Matrix:"
> print(confusion_matrix_nb)
            Actual
Predicted    setosa versicolor virginica
  setosa         15          0         0
  versicolor      0         13         2
  virginica       0          2        13
> accuracy_nb <- sum(diag(confusion_matrix_nb)) / sum(confusion_matrix_nb)
```

```
> print(paste("Naive Bayes Accuracy:", accuracy_nb))
[1] "Naive Bayes Accuracy: 0.911111111111111"
> plot_nb <- ggplot(test_data, aes(Petal.Length, Petal.Width, color = nb_predictions)) +
+   geom_point(size = 3) +
+   labs(title = "Naive Bayes Classification",
+        x = "Petal Length",
+        y = "Petal Width") +
+   theme_minimal()
> print(plot_nb)
```



RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 6A DATE :	**Implementation Decision tree for classification using sklearn and its parameter tuning.**

AIM

To Implement the Decision tree for classification using sklearn and its parameter tuning.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Create a dataset by using MS Excel and save it as .csv.

Step 3 : Read the dataset into R.

Step 4 : Examine the structure and summary statistics of the dataset.

Step 5 : Install the libraries rpart and rpart.plot.

Step 6 : Use the rpart() to fit the model.

Step 7 : Obtain and interpret the summary of the fitted model.

Step 8 : Plot the decision tree model along with the data.

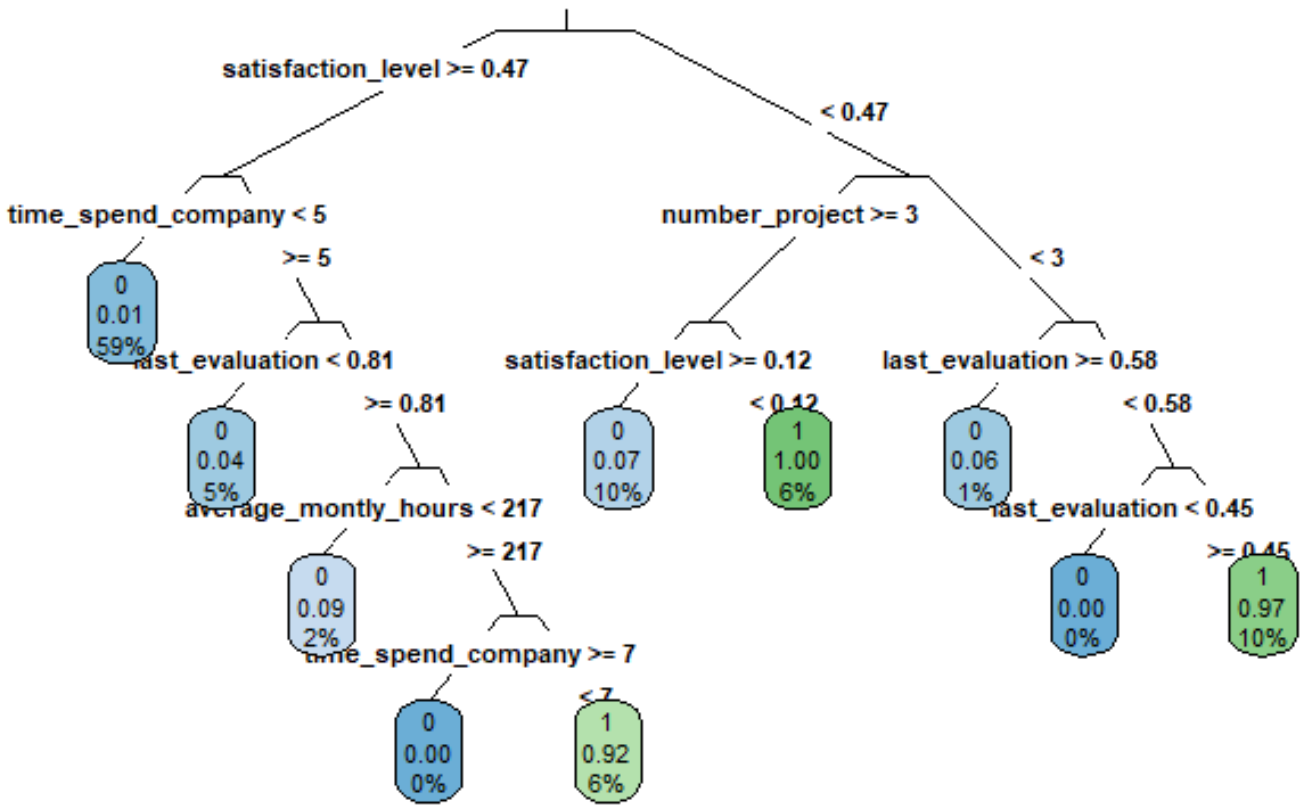
Step 9 : Save the program by using .R extension and run the program.

PROGRAM

```
library(rpart)

library(rpart.plot)

HR <- read.csv("C:/Users/ELCOT/Desktop/HR_comma_sep.csv")

Myresults <- rpart(left ~ ., method = "class", data = HR)

rpart.plot(Myresults, type = 3, fallen.leaves = FALSE, cex = 0.7)

HRtest <- read.csv("C:/Users/ELCOT/Desktop/HR_comma_sep.csv")

P1 <- predict(Myresults, newdata = HRtest, type = "class")

A1 <- as.integer(HRtest$left)

mean(P1 == A1)
```

OUTPUT

```
> library(rpart)
> library(rpart.plot)
> HR <- read.csv("C:/Users/ELCOT/Desktop/HR_comma_sep.csv")
> Myresults <- rpart(left ~ ., method = "class", data = HR)
> rpart.plot(Myresults, type = 3, fallen.leaves = FALSE, cex = 0.7)
> HRtest <- read.csv("C:/Users/ELCOT/Desktop/HR_comma_sep.csv")
> P1 <- predict(Myresults, newdata = HRtest, type = "class")
> A1 <- as.integer(HRtest$left)
> mean(P1 == A1)
[1] 0.9711981
```



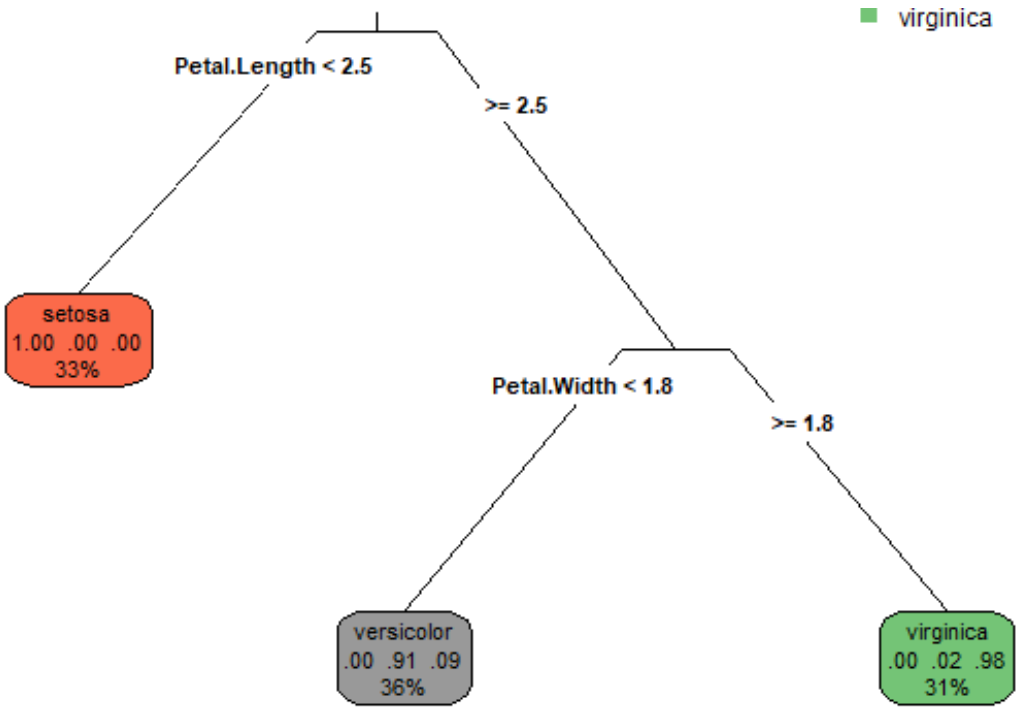
EX NO : 6B DATE :	Sample Program

PROGRAM

```
library(rpart)

library(rpart.plot)

data(iris)

model <- rpart(Species ~ ., method = "class", data = iris)

rpart.plot(model, type = 3, fallen.leaves = FALSE, cex = 0.7)

predictions <- predict(model, newdata = iris, type = "class")

actual <- iris$Species

accuracy <- mean(predictions == actual)

print(paste("Accuracy:", accuracy))
```

OUTPUT

```
> library(rpart)
> library(rpart.plot)
> data(iris)
> model <- rpart(Species ~ ., method = "class", data = iris)
> rpart.plot(model, type = 3, fallen.leaves = FALSE, cex = 0.7)
> predictions <- predict(model, newdata = iris, type = "class")
> actual <- iris$Species
> accuracy <- mean(predictions == actual)
> print(paste("Accuracy:", accuracy))
[1] "Accuracy: 0.96"
```



RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 7A **DATE :**	**Implement the K-means algorithm.**

AIM

To Implement the k-means algorithm.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Create a dataset by using MS Excel and save it as .csv.

Step 3 : Read the default dataset iris into R.

Step 4 : Examine the structure and summary statistics of the dataset.

Step 5 : Install the libraries ClusterR, cluster.

Step 6 : Use the clusplot() to fit the model.

Step 7 : Obtain and interpret the summary of the fitted model.

Step 8 : Plot the binary classification model along with the data.

Step 9 : Save the program by using .R extension and run the program.

PROGRAM

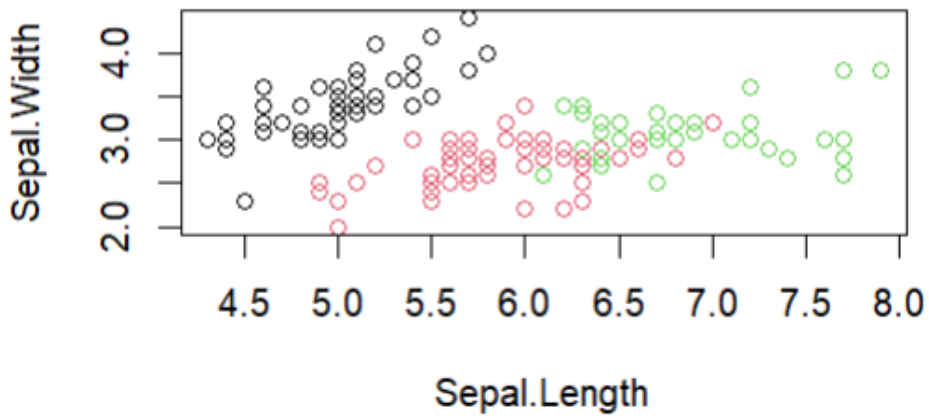
```
library(ClusterR)
library(cluster)
iris_1 <- iris[, -5]
set.seed(240)
kmeans.re <- kmeans(iris_1, centers = 3, nstart = 20)
kmeans.re
kmeans.re$cluster
cm <- table(iris$Species, kmeans.re$cluster)
cm
plot(iris_1[c("Sepal.Length", "Sepal.Width")])
plot(iris_1[c("Sepal.Length", "Sepal.Width")],
    col = kmeans.re$cluster)
plot(iris_1[c("Sepal.Length", "Sepal.Width")],
    col = kmeans.re$cluster,
    main = "K-means with 3 clusters")
kmeans.re$centers
kmeans.re$centers[, c("Sepal.Length", "Sepal.Width")]
points(kmeans.re$centers[, c("Sepal.Length", "Sepal.Width")],
       col = 1:3, pch = 8, cex = 3)
y_kmeans<- kmeans.re$cluster
```

```
clusplot(iris_1[, c("Sepal.Length", "Sepal.Width")],
        y_kmeans,
        lines = 0,
        shade = TRUE,
        color = TRUE,
        labels = 2,
        plotchar = FALSE,
        span = TRUE,
        main = paste("Cluster iris"),
        xlab = 'Sepal.Length',
        ylab = 'Sepal.Width')
```

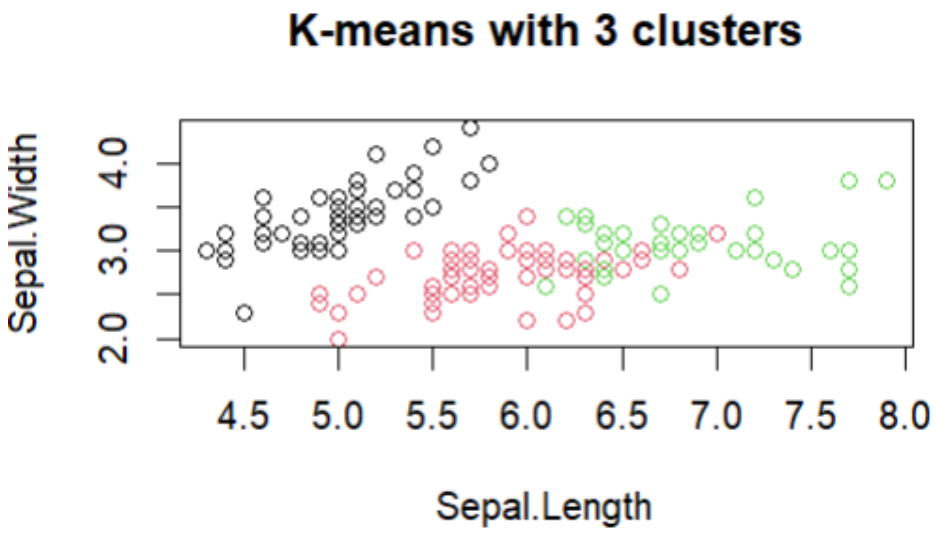
OUTPUT

```
> library(ClusterR)
> library(cluster)
> iris_1 <- iris[, -5]
> set.seed(240)
> kmeans.re <- kmeans(iris_1, centers = 3, nstart = 20)
> kmeans.re
K-means clustering with 3 clusters of sizes 50, 62, 38

Cluster means:
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1     5.006000    3.428000     1.462000    0.246000
2     5.901613    2.748387     4.393548    1.433871
3     6.850000    3.073684     5.742105    2.071053

Clustering vector:
  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [35] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [69] 2 2 2 2 2 2 2 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 2 2
[103] 3 3 3 3 2 3 3 3 3 3 3 2 2 3 3 3 3 2 3 2 3 2 3 3 2 2 3 3 3 3 3 2 3 3 3
[137] 3 3 2 3 3 3 2 3 3 3 2 3 3 2

Within cluster sum of squares by cluster:
[1] 15.15100 39.82097 23.87947
 (between_SS / total_SS =  88.4 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
```

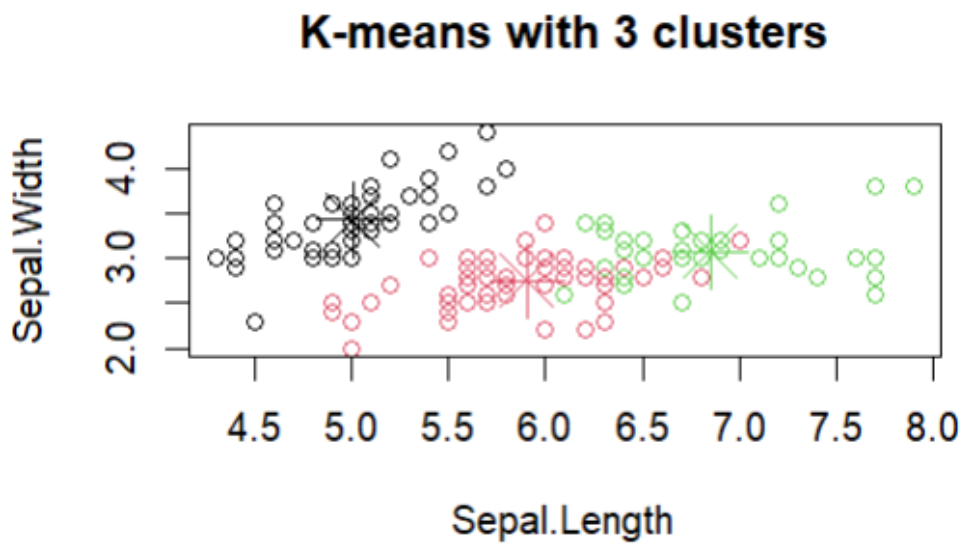
```
> kmeans.re$cluster
  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [35] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [69] 2 2 2 2 2 2 2 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 2 2 3 2
[103] 3 3 3 3 2 3 3 3 3 3 3 2 2 3 3 3 3 2 3 2 3 2 3 3 2 2 3 3 3 3 3 2 3 3 3 3 3
[137] 3 3 2 3 3 3 2 3 3 3 2 3 3 2
> cm <- table(iris$Species, kmeans.re$cluster)
> cm

            1  2  3
  FALSE     0 62 38
  TRUE     50  0  0
> plot(iris_1[c("Sepal.Length", "Sepal.Width")])
> plot(iris_1[c("Sepal.Length", "Sepal.Width")],
+      col = kmeans.re$cluster)
```



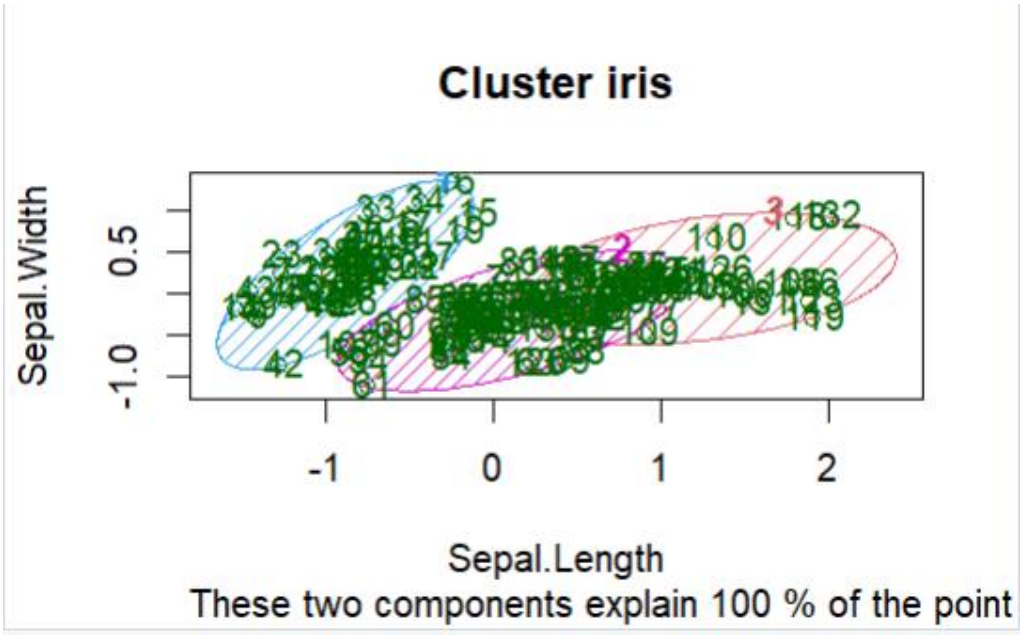

```
> plot(iris_1[c("Sepal.Length", "Sepal.Width")],
+      col = kmeans.re$cluster,
+      main = "K-means with 3 clusters")
```




```
> kmeans.re$centers
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1     5.006000    3.428000     1.462000    0.246000
2     5.901613    2.748387     4.393548    1.433871
3     6.850000    3.073684     5.742105    2.071053
> kmeans.re$centers[, c("Sepal.Length", "Sepal.Width")]
  Sepal.Length Sepal.Width
1     5.006000    3.428000
2     5.901613    2.748387
3     6.850000    3.073684
> points(kmeans.re$centers[, c("Sepal.Length", "Sepal.Width")],
+        col = 1:3, pch = 8, cex = 3)
```




```
> y_kmeans<- kmeans.re$cluster
> clusplot(iris_1[, c("Sepal.Length", "Sepal.Width")],
+         y_kmeans, +  lines = 0, + shade = TRUE,  color = TRUE, + labels = 2, + plotchar = FALSE,
+         span = TRUE,
+         main = paste("Cluster iris"),
+         xlab = 'Sepal.Length',
+         ylab = 'Sepal.Width')
```



EX NO : 7B DATE :	Sample Program

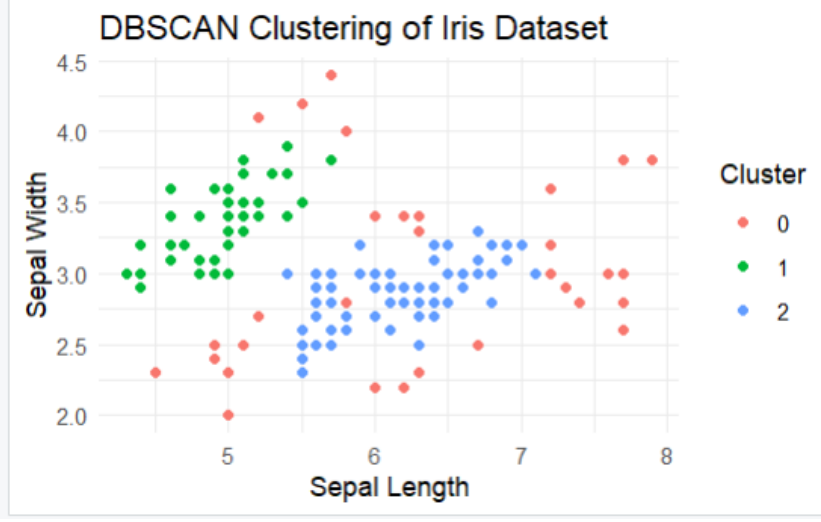
PROGRAM

```
library(dbscan)
library(ggplot2)
data(iris)
iris_1 <- iris[, -5]
iris_1_scaled <- scale(iris_1)
set.seed(240)
dbscan_result <- dbscan(iris_1_scaled, eps = 0.5, minPts = 5)
print(dbscan_result)
iris$Cluster <- factor(dbscan_result$cluster)
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Cluster)) +
  geom_point() +
  labs(title = "DBSCAN Clustering of Iris Dataset",
       x = "Sepal Length",
       y = "Sepal Width",
       color = "Cluster") +
  theme_minimal()
```

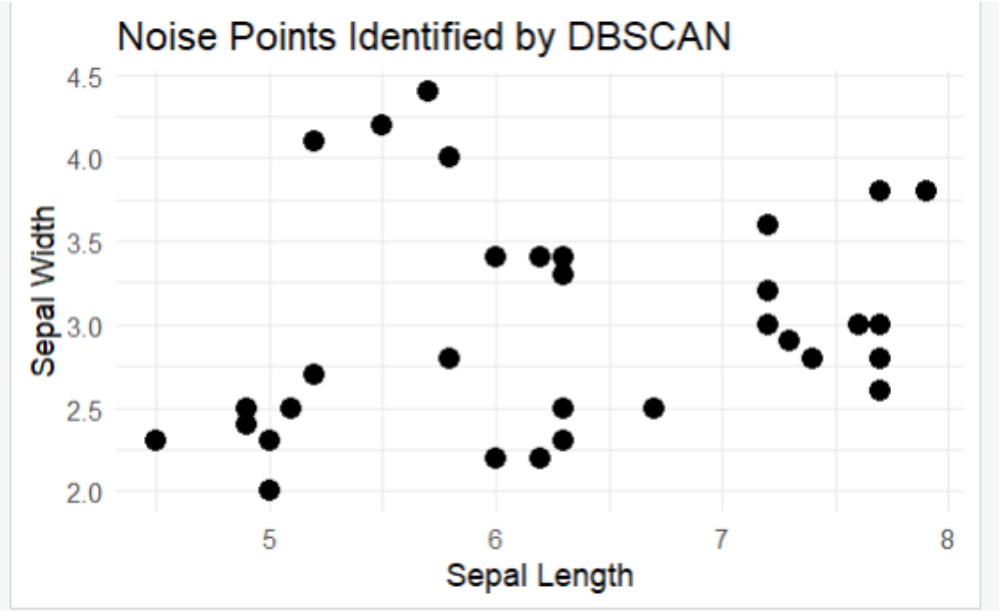
```
ggplot(iris[iris$Cluster == 0, ], aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(color = "black", size = 3) +
  labs(title = "Noise Points Identified by DBSCAN",
       x = "Sepal Length",
       y = "Sepal Width") +
  theme_minimal()
```

OUTPUT

```
> library(dbscan)
> library(ggplot2)
> print(dbscan_result)
DBSCAN clustering for 150 objects.
Parameters: eps = 0.5, minPts = 5
Using euclidean distances and borderpoints = TRUE
The clustering contains 2 cluster(s) and 34 noise points.
 0  1  2
34 45 71
Available fields: cluster, eps, minPts, metric, borderPoints
> iris$Cluster <- factor(dbscan_result$cluster)
> ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Cluster)) +
+   geom_point() +
+   labs(title = "DBSCAN Clustering of Iris Dataset",
+        x = "Sepal Length",
+        y = "Sepal Width",
+        color = "Cluster") +
+   theme_minimal()
```



```
> ggplot(iris[iris$Cluster == 0, ], aes(x = Sepal.Length, y = Sepal.Width)) +
+   geom_point(color = "black", size = 3) +
+   labs(title = "Noise Points Identified by DBSCAN",
+        x = "Sepal Length",
+        y = "Sepal Width") +
+   theme_minimal()
```



RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 8A **DATE :**	**Implement an Image Classifier using CNN in TensorFlow/Keras.**

AIM

To Implement an Image Classifier using CNN in TensorFlow/Keras.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Install the libraries Keras and Tensorflow.

Step 3 : Use the CNN() to fit the model.

Step 4 : Prepare the data and evaluate the model.

Step 5 : Plot the image classification model along with the data.

Step 6 : Save the program by using .R extension and run the program.

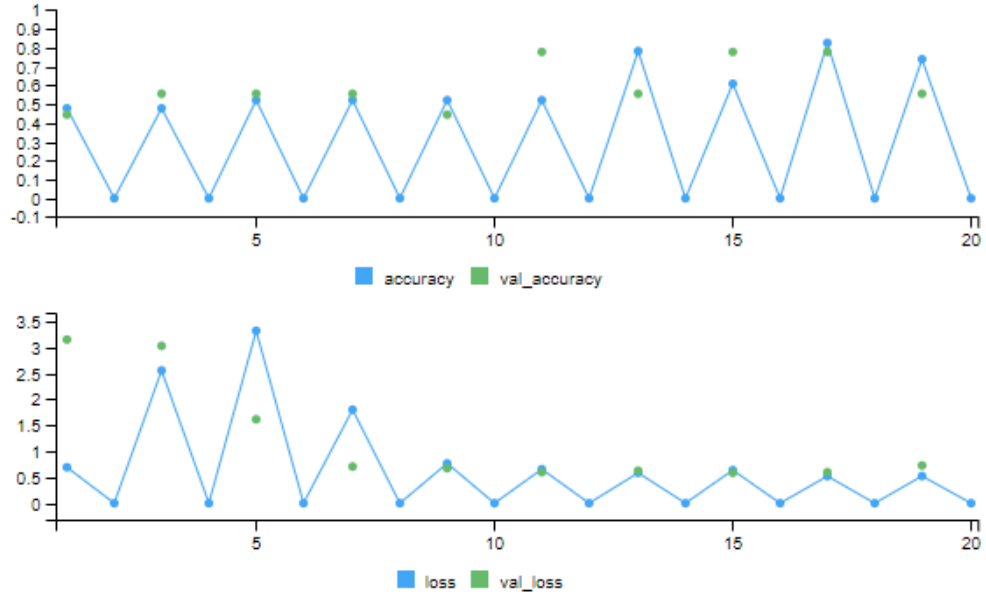
PROGRAM

```
library(keras)
install_keras()
tensorflow::tf_config()
keras::keras_version()
model <- keras_model_sequential() %>%
  layer_dense(units = 32, activation = 'relu', input_shape = c(784)) %>%
  layer_dense(units = 10, activation = 'softmax')
summary(model)
library(magrittr)
image_size <- c(150, 150)
train_datagen <- image_data_generator(
  rescale = 1/255,  shear_range = 0.2, zoom_range = 0.2, horizontal_flip = TRUE)
test_datagen <- image_data_generator( rescale = 1/255)
train_directory <- "C:\\Users\\hp\\Desktop\\Geeks images\\train"
test_directory <- "C:\\Users\\hp\\Desktop\\Geeks images\\test"
train_generator <- flow_images_from_directory(
  train_directory,
  generator = train_datagen,
  target_size = image_size,
  batch_size = 32,
```

```
  class_mode = 'binary'  )
test_generator <- flow_images_from_directory(
  test_directory,
  generator = test_datagen,
  target_size = image_size,
  batch_size = 32,
  class_mode = 'binary')
model <- keras_model_sequential() %>%
  layer_conv_2d(filters = 32, kernel_size = c(3, 3), activation = 'relu',  input_shape = c(150, 150, 3)) %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_conv_2d(filters = 64, kernel_size = c(3, 3), activation = 'relu') %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_conv_2d(filters = 128, kernel_size = c(3, 3), activation = 'relu') %>%
  layer_max_pooling_2d(pool_size = c(2, 2)) %>%
  layer_flatten() %>%
  layer_dense(units = 512, activation = 'relu') %>%
  layer_dense(units = 1, activation = 'sigmoid')
summary(model)
model %>% compile(
  optimizer = optimizer_adam(),
  loss = 'binary_crossentropy'  metrics = c('accuracy'))
```

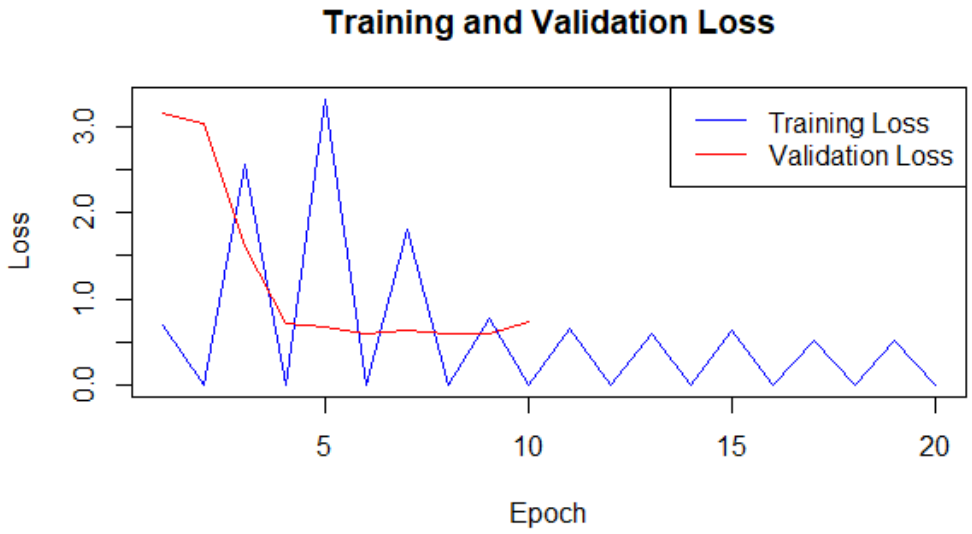
```
history <- model %>% fit(
  train_generator,
  steps_per_epoch = ceiling(23 / 32),
  epochs = 20,
  validation_data = test_generator,
  validation_steps = ceiling(9 / 32)  )
evaluation <- model %>% evaluate(
  test_generator,
  steps = as.integer(200 / 32))
cat('Test loss:', evaluation$loss, '\n')
cat('Test accuracy:', evaluation$accuracy, '\n')
plot(history$metrics$loss, type = 'l', col = 'blue', xlab = 'Epoch', ylab = 'Loss',
     main = 'Training and Validation Loss')
lines(history$metrics$val_loss, type = 'l', col = 'red')
legend('topright', legend = c('Training Loss', 'Validation Loss'),
       col = c('blue', 'red'), lty = 1)
plot(history$metrics$accuracy, type = 'l', col = 'blue', xlab = 'Epoch', ylab = 'Accuracy',
     main = 'Training and Validation Accuracy')
lines(history$metrics$val_accuracy, type = 'l', col = 'red')
legend('bottomright', legend = c('Training Accuracy', 'Validation Accuracy'),
       col = c('blue', 'red'), lty = 1)
```

OUTPUT



Test loss: 0.7260311

Test accuracy: 0.5555556



RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 9A **DATE :**	**Implement an Autoencoder in TensorFlow/Keras.**

AIM

To Implement an Autoencoder in TensorFlow/Keras.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Import necessary TensorFlow/Keras libraries.

Step 3 : Load dataset (e.g., MNIST). Normalize data between 0 and 1. Reshape the data for the autoencoder.

Step 4 : Create the **encoder** and decoder part.

Step 5 : Use a loss function like binary cross-entropy. Choose an optimizer (e.g., Adam).

Step 6 : Train the model using the prepared data and use appropriate epochs, batch size, and validation data.

Step 7 : Use the trained autoencoder to encode and decode the input data.

Step 8 : Analyze the output from the decoder to assess reconstruction quality.

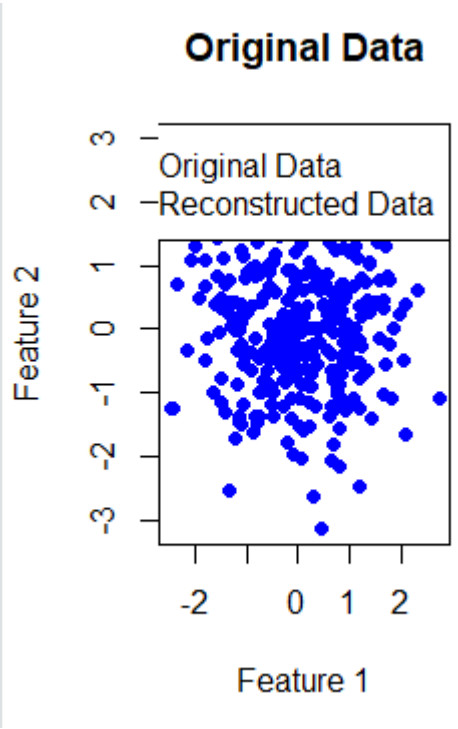
Step 9 : Save the program by using .R extension and run the program.

PROGRAM

```
library(keras)

library(tensorflow)

set.seed(123)

n <- 200

x1 <- rnorm(n, mean = 3, sd = 1)

x2 <- rnorm(n, mean = 5, sd = 1)

data <- data.frame(x1, x2)

data <- scale(data)

input_dim <- ncol(data)

encoding_dim <- 1

input_layer <- layer_input(shape = input_dim)

encoded <- layer_dense(input_layer, units = encoding_dim, activation = 'relu')  # Encoder

decoded <- layer_dense(encoded, units = input_dim, activation = 'linear')  # Decoder

autoencoder <- keras_model(input_layer, decoded)

autoencoder %>% compile(optimizer = 'adam', loss = 'mean_squared_error')

history <- autoencoder %>% fit(

  x = as.matrix(data), y = as.matrix(data),

  epochs = 100,

  batch_size = 10,

  shuffle = TRUE,
```

```
  verbose = 1
)
encoded_data <- autoencoder %>% predict(as.matrix(data))
reconstructed_data <- autoencoder %>% predict(as.matrix(data))
par(mfrow = c(1, 2))
plot(data, main = 'Original Data', xlab = 'Feature 1', ylab = 'Feature 2', col = 'blue', pch = 19)
plot(reconstructed_data, main = 'Reconstructed Data', xlab = 'Feature 1', ylab = 'Feature 2', col = 'red', pch = 19)
legend("topright", legend = c("Original Data", "Reconstructed Data"), col = c("blue", "red"), pch = 19)
```

OUTPUT


Original Data
Original Data
Reconstructed Data
Feature 2
Feature 1
3
2
1
0
-1
-2
-3
-2
0
1
2

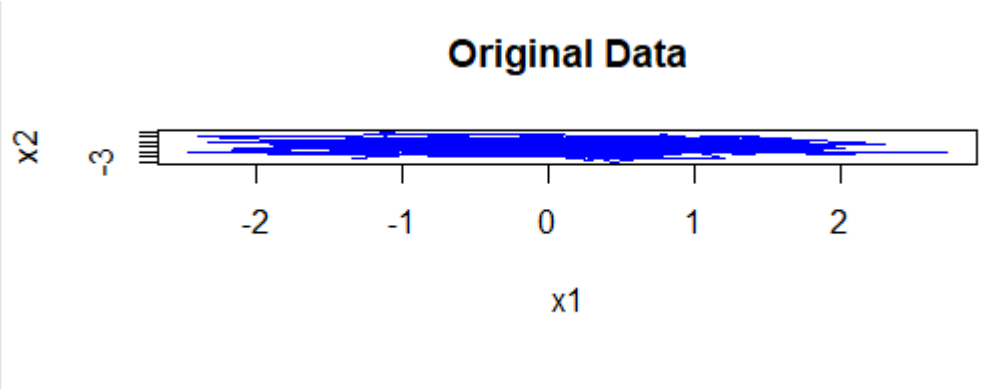
EX NO : 9B DATE :	Sample Program

PROGRAM

```
library(keras)

library(tensorflow)

set.seed(123)

data <- matrix(sin(1:100 / 5) + rnorm(100, sd = 0.1), ncol = 1)

data <- scale(data)s

input_dim <- ncol(data)

encoding_dim <- 2

input_layer <- layer_input(shape = input_dim)

encoded <- layer_dense(input_layer, units = encoding_dim, activation = "relu")

decoded <- layer_dense(encoded, units = input_dim, activation = "linear")

autoencoder <- keras_model(input_layer, decoded)

autoencoder %>% compile(optimizer = 'adam', loss = 'mean_squared_error')

history <- autoencoder %>% fit(

  x = data, y = data,

  epochs = 100,

  batch_size = 10,

  shuffle = TRUE,
```

```
  verbose = 1
)
encoder <- keras_model(input_layer, encoded)
encoded_data <- encoder %>% predict(data)
decoded_data <- autoencoder %>% predict(data)
par(mfrow = c(2, 1))
plot(data, type = 'l', col = 'blue', main = 'Original Data')
plot(decoded_data, type = 'l', col = 'red', main = 'Reconstructed Data')
```

OUTPUT



RESULT

Hence the program has been executed and the output has been verified successfully.

EX NO : 10A **DATE :**	**Implement a SimpleSTM using TensorFlow/Keras.**

AIM

To Implement a SimpleSTM using TensorFlow/Keras.

ALGORITHM

Step 1 : Open R Studio - Click File – New File – R-Script and create a new file.

Step 2 : Install the libraries Keras and Tensorflow.

Step 3: Use the () to fit the model.

Step 4 : Obtain and interpret the summary of the fitted model.

Step 5 : Plot the Simple STM model along with the data.

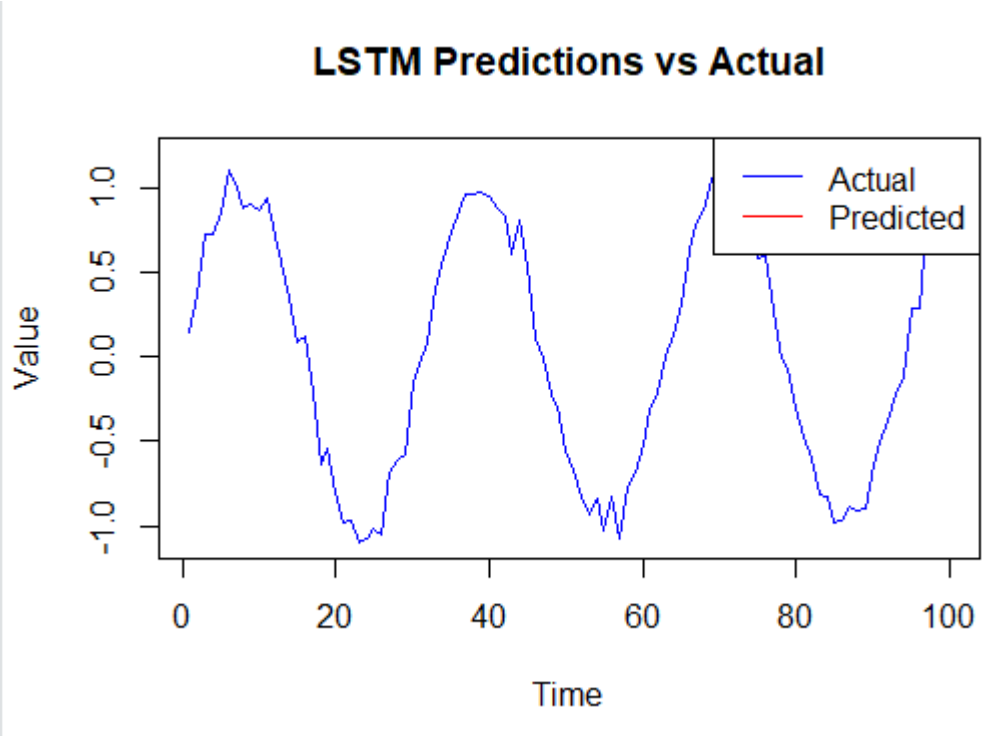
Step 6 : Save the program by using .R extension and run the program.

PROGRAM

```
library(keras)

library(tensorflow)

set.seed(123)

time <- 1:100

data <- sin(time / 5) + rnorm(100, sd = 0.1)

train_size <- 80

train_data <- data[1:train_size]

test_data <- data[(train_size + 1):100]

create_sequences <- function(data, time_steps) {

  x <- list()

  y <- list()

  for (i in seq(length(data) - time_steps)) {

    x[[i]] <- data[i:(i + time_steps - 1)]

    y[[i]] <- data[i + time_steps]}

    return(list(x = array(unlist(x), dim = c(length(x), time_steps, 1)),

                y = unlist(y)))}

time_steps <- 5

train_sequences <- create_sequences(train_data, time_steps)

test_sequences <- create_sequences(test_data, time_steps)

x_train <- train_sequences$x
```

```
y_train <- train_sequences$y

x_test <- test_sequences$x

y_test <- test_sequences$y

model <- keras_model_sequential() %>%

  layer_lstm(units = 50, input_shape = c(time_steps, 1)) %>%

  layer_dense(units = 1)

model %>% compile(loss = 'mean_squared_error', optimizer = 'adam')

model %>% fit(x_train, y_train, epochs = 100, batch_size = 10, verbose = 1)

predictions <- model %>% predict(x_test)

plot(1:length(data), data, type = 'l', col = 'blue',

     main = 'LSTM Predictions vs Actual', ylab = 'Value', xlab = 'Time')

lines((train_size + time_steps):(train_size + time_steps + length(predictions) - 1),

      c(train_data[(train_size - time_steps + 1):train_size], predictions),

      col = 'red')

legend("topright", legend = c("Actual", "Predicted"), col = c("blue", "red"), lty = 1)
```

OUTPUT


LSTM Predictions vs Actual
Actual
Predicted
Value
1.0
0.5
0.0
-0.5
-1.0
0
20
40
60
80
100
Time

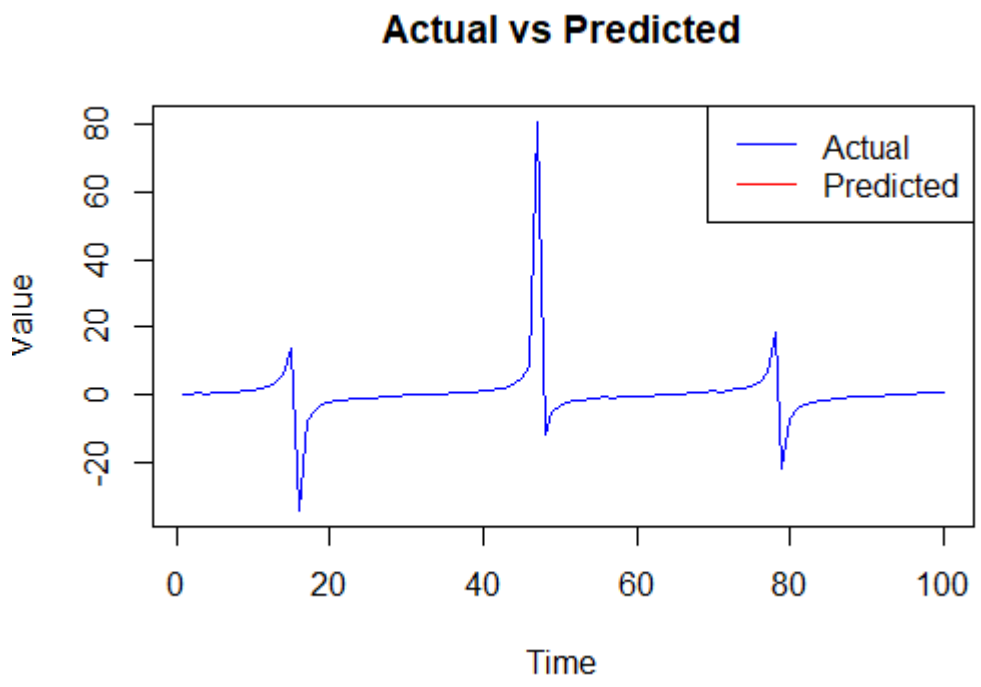
EX NO : 10B DATE :	Sample Program

PROGRAM

```
library(keras)

set.seed(123)

data <- tan(1:100 / 10) + rnorm(100, sd = 0.1)

train_data <- data[1:80]

test_data <- data[81:100]

create_sequences <- function(data, steps) {

  x <- sapply(1:(length(data) - steps), function(i) data[i:(i + steps - 1)])

  y <- data[(steps + 1):length(data)]

  list(x = array(x, dim = c(ncol(x), steps, 1)), y = y)

}

sequences <- create_sequences(train_data, 5)

x_train <- sequences$x

y_train <- sequences$y

x_test <- create_sequences(test_data, 5)$x

model <- keras_model_sequential() %>%

  layer_lstm(50, input_shape = c(5, 1)) %>%

  layer_dense(1) %>%
```

```
  compile(loss = 'mse', optimizer = 'adam')

model %>% fit(x_train, y_train, epochs = 100, verbose = 0)

predictions <- model %>% predict(x_test)

plot(data, type = 'l', col = 'blue', main = 'Actual vs Predicted', ylab = 'Value', xlab = 'Time')

lines(81:100, predictions, col = 'red')

legend("topright", legend = c("Actual", "Predicted"), col = c("blue", "red"), lty = 1)
```

OUTPUT



RESULT

Hence the program has been executed and the output has been verified successfully.